Programme de formation — Sécurité applicative pour développeurs

📁 Formateur : Quentin Danneville
Durée de la formation : 21 heures

Maîtriser l'OWASP Top 10:2025 en environnement Node.js / TypeScript

Durée	21 heures — 3 jours consécutifs de 7 heures
Modalité	Présentiel, en salle équipée — formation-action sur poste de travail
Effectif	De 6 à 12 participants (12 maximum pour garantir l'accompagnement individuel en laboratoire)
Langue	Français (supports techniques et outillage en anglais)
Référentiel	OWASP Top 10:2025
Environnement technique	Node.js / TypeScript. Les principes enseignés sont transposables à tout langage backend.

1. Public visé

La formation s'adresse à des professionnels du développement logiciel en activité :

- Développeurs et développeuses backend, frontend ou fullstack
- Lead developers, tech leads et référents techniques
- Architectes logiciels souhaitant ancrer leurs décisions dans des vulnérabilités concrètes
- Ingénieurs DevOps intervenant sur des chaînes de construction et de déploiement applicatives

Le dispositif est conçu pour un groupe hétérogène associant profils juniors (1 à 3 ans d'expérience) et profils seniors. Chaque exercice comporte deux niveaux de difficulté : un parcours guidé avec système d'indices progressifs, et un parcours avancé sous forme de revue de code où la vulnérabilité n'est pas annoncée. Les travaux pratiques des deux premiers modules sont réalisés en binômes mixtes.

Aucune expérience préalable en sécurité informatique n'est requise. La formation n'est pas destinée à des professionnels de la sécurité offensive, ni à un public non technique.

---

2. Prérequis

Prérequis professionnels

- Pratique courante du développement d'applications web ou d'API en contexte professionnel
- Maîtrise des fondamentaux du protocole HTTP : méthodes, en-têtes, codes de statut, cookies
- Capacité à lire et écrire du JavaScript ou du TypeScript ; la connaissance approfondie de l'écosystème Node.js n'est pas exigée
- Utilisation autonome de Git en ligne de commande : branches, commits, lecture d'un diff
- Notions élémentaires de bases de données relationnelles et de SQL
- Capacité à lancer un environnement conteneurisé à partir d'une commande fournie ; l'expertise Docker n'est pas requise

Prérequis matériels et techniques

Chaque participant travaille sur son propre poste. Les conditions suivantes conditionnent le bon déroulement des travaux pratiques :

- Poste disposant de 8 Go de mémoire vive au minimum, 16 Go recommandés
- Droits d'administration locaux permettant l'installation et l'exécution de conteneurs
- Docker et Node.js en version courante, installés avant la formation

- Un environnement de développement intégré au choix du participant
- Un script de vérification automatisé est transmis une semaine avant la session ; son exécution et le retour du résultat au formateur conditionnent l'accès aux travaux pratiques

Un environnement de secours accessible par navigateur est prévu pour tout participant dont le poste ne satisferait pas ces conditions.

---

3. Objectifs pédagogiques

À l'issue de la formation, le participant sera capable de :

1. Situer les dix catégories de l'OWASP Top 10:2025 et les relier aux surfaces d'attaque concrètes d'une application web
2. Reproduire une vulnérabilité en environnement contrôlé et en expliquer le mécanisme sous-jacent
3. Corriger un code vulnérable en sélectionnant la contre-mesure adaptée au contexte technique
4. Écrire un test automatisé de non-régression de sécurité et l'intégrer à une suite de tests existante
5. Conduire une revue de code orientée sécurité sur une demande d’un pull/merge request
6. Évaluer le niveau de risque associé aux dépendances d'un projet et à sa chaîne d'approvisionnement (CI/CD)
7. Intégrer des outils d'analyse statique, d'analyse de composition et de détection de secrets dans une chaîne d'intégration continue
8. Conduire une analyse de menaces simplifiée sur une fonctionnalité en phase de conception

---

4. Compétences visées

La formation vise l'acquisition d'une compétence opérationnelle transposable immédiatement : identifier, corriger et prévenir par le test les vulnérabilités les plus fréquentes des applications web. L'objectif n'est pas la sensibilisation mais l'autonomie technique — le participant repart avec des correctifs qu'il a écrits

lui-même, les tests qui les protègent, et une grille d'audit applicable à son propre code.

---

5. Programme détaillé

M0 — Fondations, modèle mental et mise en route

2 h — Jour 1 matin

Notions abordées :

- Panorama de l'OWASP Top 10:2025 : méthodologie de construction, évolutions par rapport à la version 2021, mapping pour les référentiels internes encore alignés sur 2021
- Vocabulaire opérationnel : vulnérabilité, exploit, surface d'attaque, vecteur, impact, exploitabilité
- Confidentialité, intégrité, disponibilité appliquées à des décisions de code concrètes
- Notion de frontière de confiance et de donnée non fiable
- Threat modeling léger : diagramme de flux de données, méthode STRIDE simplifiée, application à une fonctionnalité réelle
- Cadre légal et déontologique : périmètre d'intervention autorisé, interdiction de test hors périmètre
- Prise en main de l'environnement de laboratoire et de la boucle de travail Exploiter / Comprendre / Corriger / Prouver

M1 — Contrôle d'accès (A01)

4 h — Jour 1

Notions abordées :

- Distinction authentification / autorisation : deux problèmes, deux emplacements dans le code
- Référence directe non sécurisée (IDOR) : détection, exploitation, correction
- Contrôle d'accès horizontal (entre pairs) et vertical (élévation de privilège)
- Mass assignment : affectation en masse depuis le corps de requête, listes d'autorisation de champs

- Sur-exposition de données : projection de colonnes, objets de transfert (DTO), sérialisation maîtrisée
- Falsification de requête côté serveur (SSRF, intégrée à A01 en 2025) : accès aux services internes et aux métadonnées d'instance cloud, stratégies de filtrage d'URL et leurs contournements
- Principes de conception : refus par défaut, centralisation des règles, test d'autorisation systématique
- Écriture de tests d'autorisation automatisés et lecture de la différence 403 / 404

M2 — Injections et traitement des entrées (A05)

3 h — Jour 2 matin

Notions abordées :

- Mécanisme commun à toutes les injections : confusion entre code et données
- Injection SQL : requêtes paramétrées, limites structurelles du paramétrage (identifiants de colonnes, clauses de tri), listes d'autorisation
- Injection SQL via un ORM : ce que l'outil protège réellement, et où l'interpolation réapparaît
- Injection de commande système lors de l'appel à des binaires externes
- Pollution de prototype : mécanisme propre à JavaScript, fonctions de fusion d'objets, impacts
- Cross-Site Scripting stocké et réfléchi : échappement contextuel, Content Security Policy
- Validation en entrée et encodage en sortie : deux responsabilités distinctes et complémentaires
- Analyse statique de code (SAST) appliquée à la détection d'injections

M3 — Authentification et gestion de session (A07)

3 h — Jour 2

Notions abordées :

- Cycle de vie d'une session : création, transport, renouvellement, invalidation

- Cookies de session : attributs HttpOnly, Secure, SameSite et leurs effets réels
- Jetons JWT : structure, signature, vérification ; pièges du décodage sans vérification et de l'algorithme non contraint
- Stockage des jetons côté client : arbitrage entre les options et conséquences en cas de XSS
- Hachage de mots de passe : algorithmes adaptés, paramètres de coût, migration d'un existant obsolète
- Fixation de session, rotation à l'élévation de privilège
- Énumération de comptes par les messages et les temps de réponse
- Limitation de débit, verrouillage progressif, protection des parcours de réinitialisation
- Authentification multifacteur : principes, coût d'intégration, points de vigilance

M4 — Configuration, secrets et cryptographie (A02, A04)

3 h — Jour 2 après-midi

Notions abordées :

- Durcissement de la couche HTTP : en-têtes de sécurité, suppression des signatures serveur
- CORS : modèle de sécurité du navigateur, erreurs de configuration classiques, cas des requêtes authentifiées
- Séparation des environnements et valeurs par défaut dangereuses
- Gestion des secrets : variables d'environnement, coffres, rotation, cloisonnement
- Détection de secrets dans le code et dans l'historique Git ; procédure de révocation
- Cryptographie appliquée : distinction hachage / chiffrement / encodage, algorithmes obsolètes à identifier
- Génération d'aléa : différence entre pseudo-aléatoire et aléa cryptographique, impact sur les jetons

- Protection des données en transit et au repos, minimisation de la donnée collectée

M5 — Chaîne d'approvisionnement logicielle et intégrité (A03, A08)

3 h — Jour 3 matin

Notions abordées :

- Anatomie d'un arbre de dépendances : dépendances transitives, rôle et intégrité du fichier de verrouillage
- Scripts de cycle de vie des paquets et exécution de code à l'installation
- Vecteurs d'attaque : typosquatting, confusion de dépendances, compromission de mainteneur, paquet abandonné
- Analyse de cas réels documentés dans l'écosystème JavaScript
- Analyse de composition logicielle (SCA) : outils, gestion du volume d'alertes, priorisation par exploitabilité réelle
- Nomenclature logicielle (SBOM) : contenu, production, usages
- Provenance et signature des artefacts publiés
- Sécurité de la chaîne d'intégration continue : périmètre des jetons, épinglage des dépendances d'exécution, isolation des exécuteurs
- Vérification d'intégrité des données entrantes : signature de webhooks, validation des sources externes

M6 — Journalisation, alerte et gestion des échecs (A09, A10)

1 h 30 — Jour 3

Notions abordées :

- Que journaliser, à quel niveau, et surtout ce qu'il ne faut jamais écrire dans un journal
- Journal d'audit des actions sensibles : contenu minimal, immuabilité, traçabilité
- De la journalisation à l'alerte : seuils, signaux exploitables, coût du bruit
- Gestion des conditions exceptionnelles : comportement en échec ouvert (fail-open) contre échec sécurisé (fail-secure)

- Fuites d'information par les messages et traces d'erreur renvoyés au client
- Spécificités de l'exécution Node.js : rejets de promesse non gérés, exceptions non capturées, impact sur la disponibilité
- Corrélation des événements et capacité à reconstituer un incident

M7 — Synthèse : audit d'une application et transposition

1 h 30 — Jour 3 après-midi

Notions abordées :

- Épreuve pratique en équipes : identification, exploitation et correction de vulnérabilités sur une application non étudiée pendant la formation
- Construction et utilisation d'une grille d'audit de sécurité réutilisable en contexte professionnel
- Revue de code orientée sécurité : méthode de lecture, points de contrôle prioritaires
- Restitution collective et correction commentée
- Plan d'action individuel : trois mesures applicables sur le projet du participant dans le mois suivant la formation

---

6. Modalités pédagogiques

La formation repose sur une pédagogie active. La répartition du temps est la suivante :

- Environ 35 % d'apports théoriques et de démonstrations commentées par le formateur
- Environ 55 % de travaux pratiques individuels ou en binômes sur environnement de laboratoire
- Environ 10 % d'évaluations formatives, de restitutions et de temps de synthèse

Chaque vulnérabilité étudiée suit une séquence identique en quatre temps :

1. **Exploiter** — le participant reproduit l'attaque en environnement isolé
2. **Comprendre** — le mécanisme est décortiqué, y compris les raisons pour lesquelles le code vulnérable paraît correct

3. **Corriger** — le participant modifie le code lui-même, avec un système d'indices à trois niveaux
4. **Prouver** — le participant écrit le test automatisé qui échoue sur le code vulnérable et réussit après correction

La quatrième étape constitue le cœur du dispositif : elle transforme une démonstration en réflexe professionnel réutilisable dans les projets du participant.

---

7. Moyens et supports

- Un laboratoire complet fourni sous forme de dépôt Git : application volontairement vulnérable, environnements conteneurisés prêts à l'emploi, jeux de données réalistes
- Une documentation pédagogique en ligne structurée par module, conservée par le participant à l'issue de la formation
- Des environnements de démonstration isolés permettant de rejouer chaque scénario en local, y compris après la formation
- Une infrastructure mutualisée hébergeant les composants trop lourds pour un poste individuel ainsi que l'épreuve finale
- Des supports de présentation, des schémas de mécanisme et une grille d'audit imprimable
- Un kit hors ligne permettant le déroulement des travaux pratiques en cas d'indisponibilité de la connexion

L'ensemble des environnements de laboratoire est cloisonné sur un réseau interne. Aucun système externe n'est sollicité. Un rappel du cadre légal et déontologique est effectué en ouverture de session.

---

8. Modalités d'évaluation et de suivi

En amont

- Questionnaire de positionnement adressé avant la session : expérience, technologies pratiquées, exposition antérieure aux sujets de sécurité
- Vérification technique du poste de travail par script automatisé

Pendant la formation

- Évaluation formative en fin de chaque module : questionnaire interactif de cinq questions
- Évaluation continue de la production : le différentiel de code de chaque participant et les tests de sécurité qu'il rédige sont examinés par le formateur
- Point de progression individuel en fin de première et de deuxième journée

En fin de formation

- Épreuve pratique finale : audit d'une application non étudiée, en équipes, évaluée sur grille (identification, exploitation, correction, qualité du test associé)
- Questionnaire de synthèse couvrant l'ensemble des catégories du référentiel
- Questionnaire de satisfaction à chaud
- Remise d'une attestation individuelle de fin de formation mentionnant les objectifs atteints

À distance

- Enquête de transposition à trois mois : mesure de la mise en œuvre effective du plan d'action individuel

---

9. Profil de l'intervenant

Formation conçue et animée par un ingénieur logiciel disposant de plus de dix ans d'expérience en développement d'applications, également formateur en génie logiciel. L'animation s'appuie sur des pratiques de développement professionnel courantes et sur des cas réels documentés.

Version du programme : août 2026. Référentiel OWASP Top 10:2025.